

VokVision: Interim Presentation

Monocular Multi-view 3D Reconstruction Pipeline

Abhigyan Raj Mantavya Kaushik Shubhi Jain

Engineering Track

April 2026

1) Problem Statement, Scope, and Users

Problem Build an end-to-end system where users capture multi-view RGB photos and receive a downloadable 3D model.

Input Monocular image set (JPEG/PNG) captured from multiple viewpoints.

Output Reconstructed artifact exported as `point_cloud.ply` (and Gaussian model artifacts).

Target Users Students, creators, and rapid-prototyping teams needing capture-to-3D without specialized sensors.

Current Scope

- Mobile capture + backend asynchronous reconstruction.
- Monocular-only; quality depends on viewpoint coverage.
- Object isolation works with automatic fallback if SAM2 is unavailable.

2) Related Work and Baselines (I)

Classical Geometry Baseline: COLMAP-style SfM

- Strength: robust camera pose + sparse geometry on many datasets.
- Method gap: feature matching can fail on low-texture / repetitive regions.
- Method gap: hand-crafted matching + geometric verification can be brittle under motion blur.

Neural View Synthesis Baseline: NeRF

- Strength: high-fidelity novel-view synthesis.
- Method gap: slow training/inference for interactive product workflows.
- Method gap: optimization can be unstable when capture viewpoints are sparse/non-uniform.

2) Related Work and Baselines (II)

Modern 3D Priors: DUS_t3R / MAS_t3R

- Key idea: infer geometry/matching directly in 3D rather than only 2D descriptors.
- Strength: better scene-level geometric grounding for challenging correspondence.
- Method gap: quality still sensitive to image coverage and camera path diversity.

Rendering Baseline: 3D Gaussian Splatting

- Key idea: explicit Gaussian primitives for faster rendering than NeRF.
- Strength: near real-time rendering after training.
- Method gap: densification/opacity heuristics can be scene-sensitive and error-prone.

Chosen baselines for this project

- COLMAP-style SfM
- NeRF-style neural rendering
- 3D Gaussian Splatting target pipeline

3) Dataset and Evaluation Metrics

Dataset snapshot (current interim experiments)

- Latest validated run: 22 JPEG images (project: test_project_001).
- Workspace assets: gs_dataset/images and gs_dataset/sparse.
- Practical issue: small internal set, limited diversity across devices/scenes.

Evaluation Metrics

- Reconstruction quality: PSNR, SSIM, LPIPS.
- Geometry quality: Chamfer distance / completeness (when GT available).
- System metrics: end-to-end runtime, GPU VRAM, CPU RAM, storage footprint.
- Product metrics: successful job rate, average time-to-result, API error rate.

Resource relevance GPU memory and storage throughput are the dominant constraints.

4) System Overview and Current Implementation

Current end-to-end pipeline

- 1 Flutter app captures and uploads image set.
- 2 FastAPI receives files and starts background job.
- 3 Stage 1: MAST3R mapping (poses + geometry).
- 4 Stage 2: object point-cloud isolation (SAM2 or fallback) + dataset prep.
- 5 Stage 3: OpenSplat training and model export.

Completed components

- Upload/status/download API flow.
- Pipeline shell with stage orchestration.
- Integration of MAST3R, isolation stage, and OpenSplat codebases.

4) Baseline Results and Status Analysis

Module-wise interim status

Module	Interim status
Local orchestration	Functional (22 images, 2500 iterations)
MASt3R Stage	Completed (2,181,208 pts written)
Isolation Stage	Completed (final 8,294 clean pts)
OpenSplat Stage	Completed (2500 steps)
Output artifact	Generated: model.ply (934.2 KB)

Observation Current bottlenecks are environment/dependency robustness and benchmark depth, not core pipeline completion.

4) Error Analysis, Failure Cases, and Insights

Observed failure modes

- VLM branch skipped because `GEMINI_API_KEY` was missing.
- SAM2 module unavailable, forcing GrabCut+HSV fallback segmentation.
- MAST3R used slow RoPE2D PyTorch fallback (missing CUDA-compiled extension).
- HF Hub requests unauthenticated (no `HF_TOKEN`), affecting reliability under load.

Likely causes (method + system)

- Methodology: segmentation quality is sensitive when fallback replaces learned masks.
- System: optional runtime dependencies and tokens are not validated pre-run.
- System: coarse status reporting still slows root-cause diagnosis.

Engineering implication Need strict preflight checks, richer telemetry, and reproducible environment locking.

5) Individual Contributions So Far

Abhigyan Raj

- Core reconstruction integration work and pipeline-level debugging.

Mantavya Kaushik

- Backend/API orchestration and stage execution path setup.

Shubhi Jain

- Workflow validation, reporting, and integration/testing coordination.

Team-level completed milestone Interim end-to-end architecture assembled, with clear blocker localization.

5-6) Next Steps, Task Assignment, and Demo Plan

Next steps (near-term)

- Wire this successful local config into API-triggered production path.
- Add explicit job states: queued/running/failed/succeeded.
- Add metric logging for runtime, memory, and quality.
- Add preflight checks for keys/modules (GEMINI_API_KEY, HF_TOKEN, SAM2).
- Expand dataset diversity for stronger EDA and stress tests.

Future ownership

- Abhigyan Raj: reconstruction-core fixes + quality analysis.
- Mantavya Kaushik: backend reliability + status/error API enrichment.
- Shubhi Jain: experimental protocol, result synthesis, and presentation/demo flow.

Demo plan Run a 22-image job, show stage-wise logs, and download/render the generated model.ply.

Backup: Compute Requirements

- GPU: CUDA-capable, preferably at least 12GB VRAM.
- CPU RAM: 16–32GB recommended.
- Storage: SSD required for I/O-heavy training workflow.
- Operational need: queue-based scheduling for multi-user load.

Backup: Known Technical Debt

- Optional dependencies/tokens are not enforced before run start.
- Coarse status model (no explicit failed state).
- Deprecated package/runtime warnings (`google.generativeai`, Python 3.10 horizon).
- Missing CUDA-compiled RoPE2D extension causes slower MAST3R execution.